Practical - 14

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

SEMESTER: 3RD, 2026-27

ID: 25DCE110

Practical Title: Admin Dashboard with Role-Based Access Control, Dynamic Menu, and Audit Log Develop an admin dashboard that displays user statistics, student count, event count and latest registrations. Restrict access to admin users only and show dynamic menu options based on role**.**

TheoryAnswers to key questions

1. Are dashboard values loaded dynamically from the database?

Yes. SQL aggregate functions such as COUNT() retrieve the current number of users, students, and events from the database.

2. Is access restricted to admin users?

Yes. PHP session validation and role checks prevent non-admin users from accessing the admin dashboard.

3. Are role-based menus working?

Yes. Menu options are displayed according to the logged-in user's role.

4. Are important admin actions recorded in audit_logs?

Yes. Important actions, such as adding or deleting records, can be stored with the user's ID, action description, and timestamp.

Implementation/ Output/Result

Tags:

Demo code:

```
<?php
require_once __DIR__ . "/../api/auth.php";
require_once __DIR__ . "/../api/db_mysqli.php";

requireRole("admin");

function getCount($conn, $table) {
    $allowed = array("users", "students", "events");

    if (!in_array($table, $allowed, true)) {
        throw new RuntimeException("Invalid table.");
    }

    $result = $conn->query("SELECT COUNT(*) AS total FROM `$table`");
    return (int) $result->fetch_assoc()["total"];
}

$userCount = getCount($conn, "users");
$studentCount = getCount($conn, "students");
$eventCount = getCount($conn, "events");
?>

<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <title>Admin Dashboard</title>
</head>
<body>
    <h1>Admin Dashboard</h1>

    <p>Total Users: <?= $userCount ?></p>
    <p>Total Students: <?= $studentCount ?></p>
    <p>Total Events: <?= $eventCount ?></p>

    <nav>
        <a href="admin-dashboard.php">Dashboard</a>
        <a href="../api/logout.php">Logout</a>
    </nav>
</body>
</html>
```

Implementataion:

- Created an admin dashboard interface.
- Restricted dashboard access using PHP role-based authorization.
- Retrieved user, student, and event counts from MySQL.
- Displayed database statistics dynamically.
- Implemented role-based menu options.
- Recorded important administrative activities in the audit log.
- Integrated session management and logout functionality.

GitHub Repository Link:

https://github.com/kanan2007s-sys/Web-development-/tree/main

Learning Outcomes:

Learned to develop a secure admin dashboard with dynamic database statistics, role-based access control, and audit logging.

Faculty Remarks: